DOCUMENTATION API

Blog Backend — Node.js + Express.js + MySQL

Auteure : Sylviane YAMEOGO

Établissement 2iE

1. Présentation du projet

Ce projet est le backend d'une application de blog moderne. Il a été développé dans le cadre du cours de Développement Backend à 2iE, en utilisant Node.js, Express.js et MySQL.

Le backend expose une API REST complète qui permet au frontend Next.js de gérer dynamiquement les articles, les commentaires et les catégories. Concrètement, cela signifie que les articles ne sont plus stockés en dur dans le code frontend mais ils viennent de la base de données MySQL via cette API.

L'API gère également la sécurité : les utilisateurs peuvent créer un compte et se connecter. Un token JWT est alors généré et doit être fourni pour toute action d'écriture (créer, modifier, supprimer un article ou une catégorie). La lecture des articles, commentaires et catégories reste accessible à tous sans connexion.

Ce que fait concrètement l'API

- Elle permet de créer, lire, modifier et supprimer des articles de blog (opérations CRUD complètes).
- Elle gère les commentaires : n'importe qui peut laisser un commentaire sur un article.
- Elle expose les catégories pour classer les articles par thème.
- Elle gère l'inscription et la connexion des utilisateurs avec des mots de passe chiffrés (bcryptjs).
- Elle sécurise les routes sensibles grâce aux tokens JWT qui expirent après 2 heures.
- Elle est connectée au frontend Next.js via le mécanisme CORS.

Architecture technique

Le projet suit le patron MVC (Modèle - Vue - Contrôleur) qui sépare clairement les responsabilités :

Dossier / Fichier	Rôle
src/config/db.js	Connexion à MySQL avec un pool de connexions
src/models/	Contient toutes les requêtes SQL (une par entité)
src/controllers/	Traite les requêtes HTTP et appelle les modèles
src/routes/	Définit les URLs de l'API et les relie aux contrôleurs

src/middleware/authMiddleware.js	Vérifie la validité du token JWT avant d'accéder aux routes protégées
src/app.js	Configure Express, le CORS et branche toutes les routes
src/server.js	Démarre le serveur sur le port défini dans .env
.env	Variables d'environnement (port, base de données, secret JWT)
blog_db.sql	Script SQL de création et initialisation de la base de données
test.rest	Fichier de tests de toutes les routes avec REST Client (VS Code)

2. Base de données

La base de données s'appelle blog_db et contient 4 tables liées entre elles. Le fichier blog_db.sql permet de la recréer entièrement.

Table	Champs principaux
users	id, username, email, password (hashé)
categories	id, nom
articles	id, titre, slug, contenu, image_url, date_creation
comments	id, auteur, contenu, date

Remarque : quand un article est supprimé, tous ses commentaires sont automatiquement supprimés aussi (ON DELETE CASCADE). Quand un utilisateur est supprimé, ses articles gardent un user_id à NULL (ON DELETE SET NULL).

3. Authentification — JWT

L'API utilise des tokens JWT (JSON Web Token) pour sécuriser les routes d'écriture. Le token est généré à la connexion et doit être envoyé dans chaque requête protégée, dans le header Authorization.

Le token contient l'identifiant et le nom de l'utilisateur connecté. Il expire après 2 heures. Le mot de passe n'est jamais stocké en clair, il est hashé avec bcryptjs avant d'être sauvegardé en base.

Routes d'authentification

Méthode	URL	Accès	Description
POST	`/api/auth/register`	Public	Créer un compte
POST	`/api/auth/login`	Public	Se connecter et obtenir un token

Exemple : Inscription (register)

```
POST http://localhost:5000/api/auth/register
Content-Type: application/json

{
    "username": "sylviane",
    "email": "sylviane@2ie.com",
    "password": "mon_password_secu"
}

Réponse 201 : { "message": "Compte créé avec succès !" }
```

Exemple — Connexion (login)

```
POST http://localhost:5000/api/auth/login
Content-Type: application/json

{
    "email": "sylviane@2ie.com",
    "password": "mon_password_secu"
}

Réponse 200 : { "token": "eyJhbGci...", "username": "sylviane" }
```

4. Gestion des articles

Les articles sont le contenu principal du blog. La lecture est publique (pas besoin de token). La création, la modification et la suppression sont protégées, il faut être connecté.

Méthode	URL	Accès	Description
GET	/api/articles	Public	Récupérer tous les articles
GET	/api/articles/:id	Public	Récupérer un article par son ID
POST	/api/articles	🔒 Protégé	Créer un nouvel article
PUT	/api/articles/:id	🔒 Protégé	Modifier un article existant
DELETE	/api/articles/:id	🔒 Protégé	Supprimer un article

Exemple : Créer un article

```
POST http://localhost:5000/api/articles
Content-Type: application/json
Authorization: Bearer <token>

{
    "titre": "Découverte de Node.js",
    "slug": "decouverte-node-js",
    "contenu": "Node.js permet de construire des APIs ultra-rapides.",
```

```
    "image_url": "https://images.unsplash.com/photo-...",
    "categorie_id": 1
}

Réponse 201 : { "id": 2, "message": "Article créé !" }
```

Exemple — Réponse GET tous les articles

```
[
  {
    "id": 2,
    "titre": "Découverte de Node.js",
    "slug": "decouverte-node-js",
    "contenu": "Node.js permet de construire des APIs...",
    "image_url": "https://images.unsplash.com/...",
    "user_id": 1,
    "categorie_id": 1,
    "date_creation": "2026-04-04T10:30:00.000Z",
    "categorie_nom": "Technologie"
  }
]
```

5. Gestion des commentaires

N'importe qui peut lire ou ajouter un commentaire sur un article, sans avoir besoin d'être connecté. Si l'auteur ne renseigne pas son nom, il apparaît comme « Anonyme ».

Méthode	URL	Accès	Description
GET	`/api/comments/article/:articleId`	Public	Commentaires d'un article
POST	`/api/comments`	Public	Ajouter un commentaire

Exemple : Ajouter un commentaire

```
POST http://localhost:5000/api/comments
Content-Type: application/json

{
    "auteur": "Jean Dupont",
    "contenu": "Super article, merci beaucoup !",
    "article_id": 2
}

Réponse 201 : { "id": 1, "auteur": "Jean Dupont", "contenu": "...",
"article_id": 2 }
```

Exemple : Récupérer les commentaires

```
GET http://localhost:5000/api/comments/article/2

Réponse 200 :
[
  {
    "id": 1,
    "auteur": "Jean Dupont",
    "contenu": "Super article, merci beaucoup !",
    "article_id": 2,
    "date": "2026-04-04T11:00:00.000Z"
  }
]
```

6. Gestion des catégories

Les catégories permettent de classer les articles par thème. Elles sont créées une seule fois (Technologie, Développement Web, Vie Étudiante) via le fichier SQL. La consultation est publique, l'ajout est protégé.

Méthode	URL	Accès	Description
GET	/api/categories	Public	Récupérer toutes les catégories
POST	/api/categories	Protégé	Ajouter une nouvelle catégorie

Exemple : Toutes les catégories

```
GET http://localhost:5000/api/categories

Réponse 200 :
[
  { "id": 1, "nom": "Technologie" },
  { "id": 2, "nom": "Développement Web" },
  { "id": 3, "nom": "Vie Étudiante" }
]
```

Exemple : Ajouter une catégorie

```
POST http://localhost:5000/api/categories
Content-Type: application/json
Authorization: Bearer <token>

{
    "nom": "Intelligence Artificielle"
}

Réponse 201 : { "id": 4, "nom": "Intelligence Artificielle", "message": "Catégorie
créée !" }
```

7. Codes de réponse HTTP

Voici tous les codes que l'API peut retourner et leur signification :

Code	Signification	Exemple de cas
200 OK	Succès	Lecture d'articles ou de commentaires
201 Created	Ressource créée	Article, commentaire ou compte créé
400 Bad Request	Données manquantes	Titre ou contenu absent dans la requête
401 Unauthorized	Non authentifié	Token absent ou identifiants incorrects
403 Forbidden	Accès refusé	Token expiré ou invalide
404 Not Found	Ressource introuvable	Article avec cet ID inexistant
409 Conflict	Doublon détecté	Email ou username déjà utilisé
500 Server Error	Erreur serveur	Problème de connexion à la base de données